

01 / CLIENT RELEASE BRIEF

A small API migration. A complete handoff.

Swiftia video-to-shorts workflow · A review of the request, status and downstream delivery changes a consultant needs to explain before release.

Independent analysis of historical public source; no client relationship. No n8n workflow or live service was executed for this specimen. Provider compatibility and retry limits remain to be established.

34	4	0	22
nodes per version including 6 notes	changed nodes 6 parameter paths	changed connections	potentially affected nodes

WHAT CHANGES FOR THIS RELEASE

01 · Generate shorts generateShorts moves youtubeVideoId into options, removes videoSource and adds an empty webhook. Verify: the intended video reaches a provider-accepted request body.	02 · Render each item renderShort replaces id, target and a computed styling key with shortId and renderOptions. Verify: current-item ID and styling remain paired; serialized types are accepted.
03 · Recognize completion isCompleted ? changes type = done to the exact comparison status = COMPLETED. Verify: completion exits polling and reaches metadata generation.	04 · Handle failure isError ? changes type = error to status = FAILED. Its true branch returns to renderShort. Verify: repeated render requests follow an agreed stop policy.

The release decision goes beyond the four edits.

Unchanged connections still carry new behavior: FAILED triggers another render POST; a nonmatching completion response loops through Wait1. Before sign-off, establish the provider contract, item pairing and bounded rerender/polling policy, then verify the final upload. The 22-node set is static reachability, not observed runtime impact.

\$149

fixed-scope pilot

Your release gets an annotated change brief, tailored acceptance cases and a handoff document, with one revision. Up to 3 workflow pairs / 150 nodes across both versions. You execute the checks; implementation is separately scoped. Agree scope and date before payment.

Source: MI, © 2025 · n8n-youtube-to-shorts-workflow · MIT-licensed workflow. Full license.

Before: 5b4f23c → After: 7cd32c2 · Migration commit · Full analysis. Fixtures omit pinned data and export-instance metadata; credential/webhook identifiers are replaced consistently. Original node IDs, parameters and graph are retained.

02 / REQUESTS & COMPLETION

Prove the contract at the boundary.

Use controlled response fixtures in the target n8n environment. Capture evaluated values and branch outputs; attach evidence by case ID. Confirm the intended provider contract before accepting a payload.

Client / release owner: _____

Environment / n8n version: _____

API-1 Video request shape

Not run

INPUT preparingField.videoId = "fixture-video"; videoSource = "youtube".**CHECK** Evaluate the generateShorts JSON body. Require options.youtubeVideoId to contain the source ID, retain functionName: "VideoShorts" and the intended empty webhook. Confirm the provider accepts removal of videoSource.**EVIDENCE** Evaluated request JSON, source-item reference and agreed provider schema / response evidence.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

API-2 Two-item ID and styling pairing

Not run

INPUT Two current-item fixtures with ['data.shortcuts'].id values 42 and 43, each with a distinct styling object.**CHECK** Capture one renderShort body per item. Each shortId must come from the corresponding current_item_ref['data.shortcuts'].id; renderOptions must preserve the intended type and fields. Reject cross-paired IDs or styling.**EVIDENCE** Both evaluated bodies beside their originating item and styling fixtures.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

API-3 Serialization edge cases

Not run

INPUT Styling as an object, preset string, missing value and malformed upstream text; include an ID with a string value.**CHECK** The new template interpolates styling directly; the old one used .toJsonString(). Evaluate each combination. Record JSON validity and provider-required types; specify which inputs are supported and how unsupported inputs stop.**EVIDENCE** Input/output matrix with parsed JSON or exact error, plus the type decision for each case.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

STATUS-1 Completed work exits the loop

Not run

INPUT Controlled status response: {"status":"COMPLETED"}.**CHECK** isError ? takes false, then iscompleted ? takes true. Continue to metadata generation without another render/poll cycle for that item.**EVIDENCE** Branch trace, metadata input and per-item render/status request counts.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

Keep credentials out of the evidence packet. Record a redacted evaluated body and an execution reference rather than a credential-bearing request.

03 / FAILURE, POLLING & DELIVERY

Make the stopping rules explicit.

Establish the retry budget and timeout with the release owner. The exported graph shows cycles; it does not establish a safe runtime limit.

STATUS-2 Repeated failure has a stopping rule

Not run

INPUT Repeated controlled responses: {"status":"FAILED"}.

CHECK isError ? takes true back to renderShort. Record POST count and item identity. Accept repeat requests only within the agreed retry/stop policy; demonstrate the terminal outcome.

EVIDENCE Per-item POST trace, configured retry/timeout controls and final stop / error record.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

STATUS-3 Polling progresses to completion

Not run

INPUT {"status":"PENDING"}, then {"status":"COMPLETED"}. PENDING is a synthetic nonmatching value, not an asserted provider state.

CHECK The first response takes the poll/wait path; the second exits it. Record elapsed wait and status-request count against the intended polling policy.

EVIDENCE Ordered response / branch trace, wait timing and request count for the same item.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

STATUS-4 Malformed and legacy responses terminate

Not run

INPUT {}, {"status":null}, {"status":"completed"}, {"type":"done"}, {"type":"error"}.

CHECK Record validation errors or nonmatching routes. Establish and verify the timeout/error outcome so these responses cannot lead to indefinite polling or unintended rerenders.

EVIDENCE One result per fixture: route or error, request count, elapsed time and terminal outcome.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

OUTPUT-1 The intended shorts reach delivery

Not run

INPUT Two rendered-short fixtures with distinct IDs and controlled upload responses.

CHECK Confirm output count, metadata-to-file pairing, scheduled publication data and one intended upload per short.

EVIDENCE Reconciliation of source short → rendered file → metadata/schedule → controlled upload result.

Owner: _____ Evidence / run ID: _____

Observation / outcome: _____

Release decision · Pending execution evidence

Retry / timeout policy: _____

Decision (proceed / hold / revise): _____ Open issues: _____

Release owner: _____ Reviewer: _____ Date: _____